

Shopease E-Commerce Platform

Comprehensive System Architecture, File-by-File Breakdown & Crucial Code Explanations

Node.js / Express

MongoDB / Mongoose

EJS Server-Side Rendering

Multi-Role RBAC

Live Production on Render.com

DOCUMENT SCOPE & CONTENTS

This document provides an exhaustive, engineering-grade walkthrough of the entire Shopease codebase. Every directory, model, controller, route, middleware, view template, client-side script, stylesheet, and deployment configuration is thoroughly explained with its architectural justification and crucial code snippets.

1. Executive Summary & High-Level Architecture

Shopease is a full-stack, enterprise-patterned multi-vendor e-commerce platform built on the **Model-View-Controller (MVC)** architectural pattern. Unlike basic single-user shopping carts, Shopease supports three distinct user roles (Customer, Merchant Admin, and Supplier Partner), complete catalog management, real-time inventory adjustments, a live flash deal coupon engine, a Quick View modal system, and a live production deployment on Render.com backed by MongoDB Atlas.

Layer	Technologies Used	Key Responsibilities
Backend Runtime	Node.js (v18+), Express.js (v4.22)	HTTP routing, session auth, API endpoints, SSR view rendering, reverse proxy trust.
Database & ODM	MongoDB Atlas (Cloud), Mongoose (v8.24)	Document persistence, relational schema references (ObjectId), pre-save bcrypt hooks, connection pooling.
Session & Auth	<code>express-session</code> , <code>connect-mongo</code> , <code>bcryptjs</code>	Encrypted HTTP-only cookie sessions stored in MongoDB Atlas, salted one-way password hashing.
Frontend & Templating	EJS, <code>express-ejs-layouts</code> , Vanilla CSS3, Client-side ES6 JS	Semantic HTML5, layout inheritance, CSS custom properties, AJAX DOM hydration, HTML5 <code><dialog></code> .
Cloud Deployment	Render.com (Web Service), Git, <code>render.yaml</code> Blueprint	Automated CI/CD git-push deployments, zero-downtime health checking (<code>/health</code>), SSL termination.

2. Complete Project Directory & File Tree

```
/Users/mansabhatt/Desktop/My-Project/
├── server.js                # Core application entrypoint & server lifecycle
├── package.json             # Dependencies, engines, and run scripts
├── render.yaml              # Render.com Infrastructure-as-Code Blueprint
├── DEPLOYMENT.md            # Comprehensive cloud deployment walkthrough
├── .env / .env.example      # Environment variable configuration templates
├── src/
│   ├── config/
│   │   └── db.js            # MongoDB connection pooling & development fallback
│   ├── models/
│   │   ├── User.js         # User schema (Customer/Admin/Supplier, bcrypt hooks)
│   │   ├── Product.js      # Product catalog schema (supplier refs, images, stock)
│   │   ├── Category.js     # Category schema (slugs, icons, banners, itemCount)
│   │   └── Order.js        # Order schema (order items, tracking timeline, status)
│   ├── middleware/
│   │   ├── auth.js         # RBAC route guards (ensureAuth, ensureAdmin, ensureSupplier)
│   │   ├── cart.js         # Global cart calculator & view locals injector (res.locals)
│   │   └── errorHandler.js # 404 handler and centralized production error handler
│   ├── controllers/
│   │   ├── authController.js # Authentication, role redirection, customer/supplier signup
│   │   └── productController.js # Catalog browsing, multi-filter query, Quick View API, search
├── API
│   └── supplierController.js # Supplier dashboard, inventory list, 1-click stock adjuster,
├── orders
│   ├── adminController.js   # Executive metrics, Category Studio CRUD, supplier directory
│   ├── cartController.js    # Session cart management, coupon engine (SAVE20), checkout
│   ├── routes/
│   │   ├── authRoutes.js    # Login, register, logout endpoints
│   │   ├── productRoutes.js  # Home, shop catalog, product details, search/quickview APIs
│   │   ├── supplierRoutes.js # /supplier/* scoped portal routes
│   │   ├── adminRoutes.js    # /admin/* scoped console routes
│   │   ├── cartRoutes.js     # /cart/* cart manipulation endpoints
│   │   └── orderRoutes.js    # /checkout, /orders, /orders/:id tracking
│   └── seeds/
│       └── seed.js           # Automated seeder (8 categories, 16 products, 3 role accounts)
├── views/
│   ├── layouts/
│   │   └── main.ejs         # Base HTML document, navigation, dialog container, scripts
│   ├── partials/
│   │   ├── navbar.ejs       # Role-aware header, category sub-nav, counters, user menu
│   │   ├── footer.ejs       # Marketplace footer with links, copyright & payment badges
│   │   ├── alerts.ejs       # Toast & flash banner notifications
│   │   └── product-card.ejs  # Card component with Quick View trigger & live stock pills
│   └── pages/
│       ├── home.ejs         # Hero spotlight, 8 department cards, ticking deal countdown
│       ├── shop.ejs         # Catalog filter sidebar (categories, brands, price, in-stock)
│       ├── product-detail.ejs # Gallery, verified supplier box, on-page vendor stock controls
│       ├── cart.ejs         # Shopping cart table, promo coupon box, summary pricing
│       ├── checkout.ejs     # Shipping details, payment selector, order confirmation
│       ├── orders.ejs       # Order history & order tracking timeline view
│       ├── 404.ejs & 403.ejs # Error and unauthorized access pages
│       ├── auth/
│       │   ├── login.ejs    # 3-Way role tab switcher & 1-click demo login buttons
│       │   └── register.ejs # Customer vs Supplier Partner registration form
│       ├── admin/
│       │   └── dashboard.ejs # Executive KPIs (Gross sales, orders, suppliers, restock
├── alerts)
│   ├── products.ejs         # Admin catalog inventory with 1-click stock stepper
│   ├── categories.ejs      # Category Studio CRUD interface
│   └── suppliers.ejs        # Certified vendor partner directory
```

```

├── ┌── orders.ejs          # Global order fulfillment manager
    └── supplier/
        ├── dashboard.ejs  # Vendor operational metrics & restock warnings
        ├── products.ejs   # Vendor inventory table with 1-click +/- steppers
        ├── product-form.ejs # Add / Edit product listing form
        └── orders.ejs     # Supplier item-level fulfillment view
└── public/
    ├── css/
    │   └── main.css       # Custom CSS variables, responsive grid, glassmorphism, badges
    └── js/
        └── main.js        # AJAX Quick View fetch, stock stepper handler, countdown clock

```

3. File-by-File Technical Deep Dive & Crucial Code Snippets

3.1 `server.js` — Application Entrypoint & Server Lifecycle

Purpose: Initializes the Express server, mounts global middlewares, handles database connection, configures reverse proxy trust for Render, binds HTTP routes, and establishes graceful shutdown listeners.

`server.js` (Crucial Snippet: Reverse Proxy Trust & Session Store)

Express + MongoStore

```
// Enable reverse proxy trust for Render.com (enables secure HTTPS cookies)
app.set('trust proxy', 1);

// Express Session with MongoStore reusing active Mongoose connection
const isProduction = process.env.NODE_ENV === 'production';
app.use(
  session({
    secret: process.env.SESSION_SECRET || 'shopease_default_session_secret_key',
    resave: false,
    saveUninitialized: false,
    store: MongoStore.create({
      client: mongoose.connection.getClient(),
      ttl: 14 * 24 * 60 * 60 // 14-day persistent sessions in MongoDB Atlas
    }),
    cookie: {
      maxAge: 14 * 24 * 60 * 60 * 1000,
      httpOnly: true, // Mitigates XSS cookie theft
      sameSite: 'lax', // Protects against CSRF attacks
      secure: isProduction // Transmitted only over HTTPS in production
    }
  })
);

// Zero-downtime Health Check endpoint for Render.com orchestrator
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'ok',
    uptime: process.uptime(),
    timestamp: new Date().toISOString(),
    environment: process.env.NODE_ENV || 'development'
  });
});
```

Why this code is crucial:

- `app.set('trust proxy', 1)` : Render terminates TLS at its cloud load balancer. Express receives the connection over an internal proxy. Without this line, Express considers the connection unencrypted HTTP and refuses to set `secure` cookies, breaking logins.
- `client: mongoose.connection.getClient()` : Reuses the existing MongoDB connection pool instead of opening an unneeded second connection to MongoDB Atlas.
- `GET /health` : Queried by Render's health checker before redirecting live traffic to a new build.

3.2 `src/config/db.js` — Resilient Database Connection

Purpose: Connects to MongoDB Atlas using Mongoose with resilient fallback to local MongoDB in development if remote credentials fail.

```
const connectDB = async () => {
  const primaryUri = process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/shopease_ecommerce';
  const localFallback = 'mongodb://127.0.0.1:27017/shopease_ecommerce';

  try {
    const conn = await mongoose.connect(primaryUri);
    console.log(`[MongoDB] Connected successfully: ${conn.connection.host}/${conn.connection.name}`);
    return conn;
  } catch (error) {
    if (process.env.NODE_ENV !== 'production' && primaryUri !== localFallback) {
      console.warn(`[MongoDB] Falling back to local MongoDB: ${localFallback}...`);
      const localConn = await mongoose.connect(localFallback);
      return localConn;
    }
    throw error;
  }
};
```

3.3 Data Layer — src/models/

User.js — Multi-Role User Schema & Password Hashing

Supports three user roles: 'customer', 'admin', and 'supplier'. Contains nested `supplierInfo` for merchant business credentials.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  role: {
    type: String,
    enum: ['customer', 'admin', 'supplier'],
    default: 'customer'
  },
  supplierInfo: {
    companyName: String,
    storeName: String,
    phone: String,
    address: String,
    isVerified: { type: Boolean, default: false }
  },
  wishlist: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Product' }]
}, { timestamps: true });

// Pre-save hook: Hashes password with 10 salt rounds before saving
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Instance method to compare candidate password during login
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};
```

Why this code is crucial: Passwords are never stored in plain text. The `isModified('password')` guard ensures passwords are only hashed when created or updated, preventing double-hashing on profile edits.

Product.js — Catalog Schema with Supplier Ownership

Stores pricing, discounts, multi-angle images, specifications, ratings, and explicit relational ownership via `supplier: { type: ObjectId, ref: 'User' }`.

src/models/Product.js

Product Schema

```
const productSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  slug: { type: String, required: true, unique: true, lowercase: true },
  brand: { type: String, required: true },
  category: { type: String, required: true, index: true },
  supplier: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  supplierName: { type: String, default: 'Shopease Official' },
  price: { type: Number, required: true },
  originalPrice: { type: Number },
  discountPercent: { type: Number, default: 0 },
  stock: { type: Number, required: true, default: 0 },
  thumbnail: { type: String, required: true },
  images: [{ type: String }],
  description: { type: String, required: true },
  features: [{ type: String }],
  specs: [{ key: String, value: String }],
  ratings: { type: Number, default: 5.0 },
  numReviews: { type: Number, default: 0 },
  isFeatured: { type: Boolean, default: false }
}, { timestamps: true });
```

Category.js & Order.js

- **Category.js** : Contains `slug`, `icon` (emoji), `image` (Unsplash cover photo), `bannerImage`, `itemCount` (cached product count for O(1) rendering), and `isFeatured`.
- **Order.js** : Contains `orderItems` array (referencing `Product`), `totalPrice`, `shippingAddress`, `paymentMethod`, `orderStatus`, and an audit array `trackingEvents: [{ status, message, timestamp, location }]`.

3.4 Middleware Layer — `src/middleware/`

`auth.js` — Role-Based Access Control (RBAC) Guards

`src/middleware/auth.js` (Crucial Snippet: Role Gatekeepers)

RBAC Guards

```
// Admin Gatekeeper
const ensureAdmin = (req, res, next) => {
  if (req.session && req.session.user && req.session.user.role === 'admin') {
    return next();
  }
  return res.status(403).render('pages/403', {
    title: 'Access Denied',
    message: 'You need administrator privileges to access this console.'
  });
};

// Supplier Gatekeeper (Accessible by Suppliers and Admins)
const ensureSupplier = (req, res, next) => {
  if (req.session && req.session.user &&
    (req.session.user.role === 'supplier' || req.session.user.role === 'admin')) {
    return next();
  }
  return res.status(403).render('pages/403', {
    title: 'Access Denied',
    message: 'You need an authorized Supplier or Admin account to access this portal.'
  });
};
```

`cart.js` — Global Session Cart & View Locals Injector

Executes on **every incoming request**. It calculates live cart totals, applies coupons (e.g. `SAVE20`), enforces the \$50 free shipping threshold, and binds variables to `res.locals` so navbar badges and session state are accessible in all EJS templates without passing them manually in every controller.


```
const initCartAndLocals = async (req, res, next) => {
  if (!req.session.cart) {
    req.session.cart = { items: [], totalQty: 0, subtotal: 0, discount: 0, shipping: 0, total: 0,
coupon: null };
  }
  const cart = req.session.cart;
  let totalQty = 0, subtotal = 0;
  cart.items.forEach(i => { totalQty += i.quantity; subtotal += i.price * i.quantity; });
  cart.totalQty = totalQty;
  cart.subtotal = parseFloat(subtotal.toFixed(2));

  // 20% Promo discount calculation
  let discount = 0;
  if (cart.coupon && cart.coupon.type === 'percentage') {
    discount = (subtotal * cart.coupon.value) / 100;
  }
  cart.discount = parseFloat(Math.min(discount, subtotal).toFixed(2));

  // Free shipping over $50
  cart.shipping = (cart.subtotal > 50 || cart.subtotal === 0) ? 0 : 9.99;
  cart.total = parseFloat(Math.max(0, cart.subtotal - cart.discount + cart.shipping).toFixed(2));

  // Global variables accessible in all EJS files
  res.locals.cart = cart;
  res.locals.currentUser = req.session.user || null;
  res.locals.navCategories = await Category.find().sort({ name: 1 }).lean();
  next();
};
```

3.5 Controllers — src/controllers/

supplierController.js — 1-Click Stock Adjuster & Scoped Catalog

Suppliers can only view and mutate products they own (`supplier: req.session.user._id`). Admins can mutate any catalog item.

src/controllers/supplierController.js (Crucial Snippet: 1-Click Stock Adjustment)

Stock Increment / Decrement API

```
exports.adjustStock = async (req, res) => {
  try {
    const { id } = req.params;
    const { action, amount, stock } = req.body; // action: 'add' | 'remove' | 'set'

    // Restrict query to supplier's items unless admin
    const filter = { _id: id };
    if (req.session.user.role !== 'admin') {
      filter.supplier = req.session.user._id;
    }

    const product = await Product.findOne(filter);
    if (!product) return res.status(404).json({ success: false, message: 'Product not found or unauthorized' });

    const oldStock = product.stock;
    if (action === 'add') {
      product.stock = Math.max(0, product.stock + (parseInt(amount) || 1));
    } else if (action === 'remove') {
      product.stock = Math.max(0, product.stock - (parseInt(amount) || 1));
    } else if (action === 'set') {
      product.stock = Math.max(0, parseInt(stock) || 0);
    }
    await product.save();

    const change = product.stock - oldStock;
    const msg = `${change >= 0 ? 'Added' : 'Removed'} ${Math.abs(change)} units. New stock: ${product.stock}`;

    // Respond with JSON for AJAX or redirect for HTML form POST
    if (req.xhr || req.headers.accept?.includes('application/json')) {
      return res.json({ success: true, productId: product._id, newStock: product.stock, oldStock, change, message: msg });
    }
    res.redirect('/supplier/products?success=' + encodeURIComponent(msg));
  } catch (error) {
    res.status(500).json({ success: false, message: 'Stock update failed' });
  }
};
```

productController.js — Quick View API & Multi-Filter Engine

- **apiQuickView** : Returns a compact JSON representation of a product (all high-res image angles, live stock, specifications, verified supplier name, price) to hydrate the HTML5 `<dialog>` without page reload.
- **getShop** : Implements compound MongoDB queries with keyword search regex, category filter, brand match, price ranges (`$gte` , `$lte`), customer ratings filter, in-stock only filter (`stock: { $gt: 0 }`), and pagination.
- **apiSearch** : Provides debounced live autocomplete search over product titles and brands.

`adminController.js` — Command Center & Category Studio

- **Executive Dashboard** (`getDashboard`): Computes gross sales via MongoDB aggregation over paid orders, total orders, active catalog units, low-stock warnings (`stock <= 5`), and verified vendor partners count.
- **Category Studio**: Complete CRUD controller allowing admins to create, edit, visual preview, and delete store departments.

3.6 Frontend Views & Client-Side Interactivity

`views/partials/product-card.ejs`

Features secondary image hover transitions, dynamic stock status badges (`● In Stock` vs `⚠ Only X left` vs `× Sold Out`), verified supplier attribution (`🏪 Sold by: Apex Global Supplies`), and a floating Quick View trigger button.

```
// 1. Quick View Modal Hydration
function initQuickViewModal() {
  const modal = document.getElementById('quickViewModal');
  const modalBody = document.getElementById('quickViewModalBody');

  document.addEventListener('click', async (e) => {
    const btn = e.target.closest('.quick-view-btn');
    if (!btn) return;
    e.preventDefault();
    const productId = btn.dataset.productId;
    modal.showModal(); // Opens native HTML5 dialog with backdrop

    const res = await fetch(`/api/products/${productId}/quickview`);
    const data = await res.json();
    if (data.success) {
      // Injects multi-image thumbnail switcher, live stock pills, and AJAX add-to-cart
      renderQuickViewDOM(data.product);
    }
  });
}

// 2. Asynchronous 1-Click Stock Adjuster (Add / Remove / Set)
function initStockAdjusters() {
  document.addEventListener('click', async (e) => {
    const btn = e.target.closest('.btn-stock-adjust') || e.target.closest('.btn-stock-set');
    if (!btn) return;
    e.preventDefault();
    const widget = btn.closest('.stock-action-widget');
    const endpoint = widget.dataset.endpoint;
    const action = btn.dataset.action; // 'add' | 'remove' | 'set'
    const amount = btn.dataset.amount || null;
    const stockVal = action === 'set' ? widget.querySelector('.stock-direct-input').value : null;

    const res = await fetch(endpoint, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
      body: JSON.stringify({ action, amount, stock: stockVal })
    });
    const data = await res.json();
    if (data.success) {
      showToast(data.message, 'success');
      // Updates table stock badge pill and input without full page refresh
      updateStockBadgeDOM(data.productId, data.newStock);
    }
  });
}
```

3.7 Database Seeding & Cloud Deployment

`src/seeds/seed.js`

Automated seeder that flushes existing data and creates:

- **8 Curated Departments:** Electronics, Fashion, Footwear, Home & Living, Accessories, Beauty & Skincare, Sports & Outdoors, Workstation & Gaming.
- **3 Pre-Configured Role Accounts:**
 - `admin@store.com` / `admin123` (Role: `admin`)
 - `supplier@apex.com` / `supplier123` (Role: `supplier`)
 - `john@example.com` / `customer123` (Role: `customer`)
- **16 Multi-Image Products:** Complete with high-resolution Unsplash images, features, technical specifications, initial warehouse stock counts, and verified seller attributions.
- **Sample Orders:** Initial order items with tracking events timeline.

`render.yaml` — [Infrastructure as Code for Render.com](#)

`render.yaml` (Render Blueprint)

YAML Specification

```
services:
- type: web
  name: shopease-ecommerce
  runtime: node
  plan: free
  buildCommand: npm install
  startCommand: npm start
  healthCheckPath: /health
  autoDeploy: true
  envVars:
    - key: NODE_ENV
      value: production
    - key: PORT
      value: 10000
    - key: SESSION_SECRET
      generateValue: true
    - key: MONGODB_URI
      sync: false
```

VERIFIED PRODUCTION DEPLOYMENT

The application is live and accessible at:

<https://web-development-homework-sem-3-1.onrender.com>

The production instance runs Node.js connected to MongoDB Atlas over TLS, with zero-downtime rolling deploys triggered via GitHub push.